

A Python-based Packet Sniffer Simulation for Network Traffic Analysis

Technical Research & Live Terminal Output Report

Aryan Rekhi
E21CSEU0223

Arihant Saxena
E21CSEU0204

Ayushman Pranav
E21CSEU0245

Bonthu Sai Easwar
E21CSEU0252

Department of Computer Science Engineering, Bennett University
Course: CSET207 Computer Networks (Semester 4, B.Tech)

March 2023

1 Abstract

Welcome to our research paper introducing a Packet Sniffer Simulation for Network Traffic Analysis. In this paper, we present an avant-garde algorithm that generates random packets with different protocols, captures them, and records vital information in a data frame. Our simulation tool offers fantastic worth for a range of applications, including educational purposes, business traffic analysis, and testing network traffic analysis tools. Our simulation tool proposes a user-friendly answer for analyzing and troubleshooting network traffic issues. With our state-of-the-art algorithm, network traffic can be simulated and captured, providing a thorough analysis of the transmitted data. Furthermore, our tool is positively versatile, allowing for customized simulations tailored to satisfy specific network analysis needs. Our algorithm provides a comprehensive analysis of network traffic, making it effortless to identify and address performance issues and safety threats. Additionally, our simulation tool offers flexibility, allowing for a personalized approach to network analysis, satisfying the specific needs of each user. In summary, our simulation tool is an incredible resource for anyone interested in network analysis. Whether you are a student, a business owner, or a network analyst, our tool delivers a user-friendly solution to your network traffic analysis needs. With our cutting-edge algorithm and customizable simulations, analyzing and troubleshooting network traffic problems has never been easier.

2 Introduction

Have you ever encountered network issues that you can't appear to diagnose? Packet sniffing may be the solution you need! Packet sniffing, also known as network sniffing or packet analysis, is a technique used to capture and analyze network traffic. It implicates intercepting and examining the scopes of packets transmitted over a network to gain insight into the behavior of the network and the applications that utilize it.

Packet sniffing is a vital tool for network analysis and troubleshooting. It enables network administrators and safety professionals to recognize and diagnose problems that affect network performance or security. By studying

the packets between devices on the network, they can pinpoint the source of the problem, such as a flawed device, misconfigured network settings, or a safety breach.

In addendum to troubleshooting, packet sniffing is also helpful for network optimization and monitoring. Administrators can create educated decisions about resource allocation and network performance optimization by analyzing the traffic patterns and bandwidth usage of different applications and gizmos on the network. However, note that packet sniffing is a discreet activity that affects intercepting and analyzing potentially sensitive data sent over the network. Therefore, it should only be performed with the explicit consent of all parties concerned and in accordance with relevant laws and regulations.

That's where our project comes in! Our team has designed a Python-based network packet sniffer that thwarts and examines network traffic in real time. Our objective is to deliver network administrators and safety professionals with an assertive and adaptable tool for network analysis, troubleshooting, and optimization. Our packet sniffer seizes and disassembles packets as they are transmitted over the network, delivering helpful insights into network behavior and performance. This data helps users identify problems, optimize resources, and enhance security.

We offer a user-friendly interface and customizable choices that create our tool easy to use, even for non-experts. Whether you're a novice or a refined professional, our packet sniffer meets your needs. We used the power of Python 3 and the NET-Protocols library to deliver a high-performance and feature-rich packet sniffer. Our tool catches and analyzes network traffic, also supplies users with a prosperity of information, including packet metadata, protocol information, and packet data.

In summary, our project contributes to network analysis and troubleshooting. It simplifies and streamlines the process of network analysis and optimization. Its ease of use, customizability, and feature-rich capabilities make it an essential tool for any network administrator or security professional looking to improve their network performance and security. Try our packet sniffer today and take the first step towards mastering network analysis!

3 Literature Review

There are many studies and research available on traffic analysis of network and development of packet sniffers. Scapy is a library used for manipulation and is used in numerous studies as mentioned before to analyse and manipulate network traffic most efficiently. Packet Sniffers like Wireshark and tcpdump are used and preferred in analysis of network and research in security.

There are many other researchers who have developed multiple simulation tools to study and understand patterns in network traffic and study performance of different network protocols. Ns3 is a popular network simulator that has been used to study, analyse and optimise respective network protocols. Mininet is also widely used as a network emulator which creates a virtual network to be studied and also study and analyse behaviour of respective network protocols.

There are already tools which are valuable for traffic analysis of network, but there is still room available for improvement and development of new simulation tools. We have made a packet sniffer simulation in Python language which focuses on providing a simple, better, and with easier interface for generation and processing of

random packets in a network. Our approach aims or focuses on closing the gap between complex network simulators and packet capture tools of the real world, which makes it more accessible for users with limited network expertise.

4 Methodology

To forge a packet sniffer, we used Python and the Scapy library, which is a versatile tool for packet manipulation. The code we developed has some functions that yield random packets, process packets, and simulate a packet sniffer in action.

4.1 Packet Formation

The simulation code generates packets that follow four distinct protocols: TCP, UDP, ICMP, and ARP. An Ethernet layer, an IP layer, and a protocol-specific layer are all present in each packet. To simulate real-world network traffic, the origin and destination IP addresses, as well as the MAC addresses, are generated at random.

4.2 Packet Handling

Once formed, packets are processed by the "process-packet" function. This function captures the packet and extracts data, which is subsequently put in a DataFrame. This data includes the packet's timestamp, protocol, source and destination IP addresses, source and destination ports, and packet length. By analysing this data, network managers and security professionals can gain a thorough understanding of network traffic patterns, identify potential problems, and optimise network performance.

4.3 Packet Sniffer Simulation

The packet sniffer simulation function generates a specified number of packets and processes them. Finally, the resulting DataFrame is exported to a CSV file for further analysis.

4.4 Network Simulation Graphs

Visual representations of network traffic generated during network simulations offer a breakdown of the various network parameters. By providing visual representations of complex data, network graphs help to simplify the analysis of network traffic patterns. They can be used in various scenarios, such as during network testing, to analyze performance and identify areas of improvement. Furthermore, these graphs can be used to troubleshoot network issues, such as identifying the source of a network bottleneck or identifying potential security threats.

5 Results

The simulation engine successfully processes packet parameters and structures metadata dynamically inside a high-performance framework. The system parses structural layers into comprehensive records containing timestamps, operational protocol levels, source/destination addressing, logical ports, and total byte lengths.

5.1 Eight-Point Analysis of Output Results

1. Live Packet Capturing and Frame Structure: The terminal console environment actively records live transmission frames. The extraction sub-engine structurally parses every incoming frame directly into primary metrics, separating metadata into Source IP, Destination IP, Protocol, Packet Length, and structural hexadecimal payload formats.

2. Raw Payload Data Extraction: Upper-layer application telemetry is systematically intercepted. The capture arrays parse highly complex raw hex streams and control characters embedded inside the transport layout layer (e.g., raw application signatures mapped into binary byte array layouts like `b'\x05\x00\x00...'`).

3. Protocol Layer Identification: The transport layer validation mapping tracks logical numbered indexes natively. Active logging maps Protocol: 6 to verify active **TCP (Transmission Control Protocol)** traffic operations routing within the monitored submasks.

4. Packet Length Variable Fluctuations: Clear variation trends appear inside the packet-length parameters. Small handshake control buffers clocking at precisely **66 bytes** are clearly differentiated from bulk application packet data segments that scale up to **1330 or 1334 bytes**.

5. Local and Remote IP Resolution: Structural path tracing successfully monitors routing flows between local private spaces and external public interfaces. Captured traffic matrices register private local targets (e.g., 10.165.69.63) actively communicating across public remote blocks like 49.44.197.185 and 92.223.116.197.

6. Network Latency & Connectivity Baseline (ICMP Logs): Live connectivity baselines implemented via host ICMP diagnostic arrays (the ping utility) demonstrate a rock-solid network state across major nodes (google.com, flipkart.com, tmv.com), recording a **0%** packet loss coefficient.

7. Distinct Node Path Testing: Structural round-trip time tracking logs precise variations based on physical distance and network hops. Low-latency pathways establish tight round-trip metrics scaling near ****105ms to 109ms****, whereas high-overhead target structures average higher processing loops topping at ****297ms****.

8. Dynamic Data Visualization (Graphical Outputs): Captured telemetry variables pipe seamlessly into automated visualization toolkits to yield high-fidelity tracking analytics. The metrics track cumulative protocol distributions, packet counts grouped by source/destination, and live historical byte counts over active session timelines.

Terminal - PowerShell - Packet Sniffer Output Capture

```
Source IP      : 10.165.69.63
Destination IP : 49.44.197.185
Protocol       : 6
Packet Length  : 66
Payload        : b'E\x00\x004s\xae@\x00\x80\x06\x00\x00\n\x0a5E?
1,\xc5\xb9\xd6\x18\x00PC[\xd4\xf1gc\xca...'
=====
Source IP      : 49.44.197.153
Destination IP : 10.165.69.63
Protocol       : 6
Packet Length  : 1322
Payload        : b'E\x00\x05\x1c\x00\x00@\x004\x06\xfb21,\x99\n\xaa5E?1,\xc5\x99\xd6\x1e\x00P...'
=====
Source IP      : 10.165.69.63
Destination IP : 92.223.116.197
Protocol       : 6
Packet Length  : 78
Payload        : b'E\x00\x00@\x8b\x80\x00\x00\n\xaa5E?\xdft\xc5\xcf\xd7\x00P^\x1n\xb0\xc0\xd1...'
=====
```

Figure 1: Extracted Packet Metadata and Raw Binary Payload Shell

Command Prompt - ICMP Diagnostic Tracking (ping)

```
C:\Users\jayks> ping google.com
Pinging google.com [142.250.29.138] with 32 bytes of data:
Reply from 142.250.29.138: bytes=32 time=102ms TTL=108
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Approximate round trip times in milli-seconds: Minimum = 102ms, Maximum = 119ms, Average = 109ms

C:\Users\jayks> ping flipkart.com
Pinging flipkart.com [103.243.32.90] with 32 bytes of data:
Reply from 103.243.32.90: bytes=32 time=95ms TTL=53
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Approximate round trip times in milli-seconds: Minimum = 95ms, Maximum = 115ms, Average = 105ms
```

Figure 2: ICMP Echo Routing Control Latency and Verification Metrics

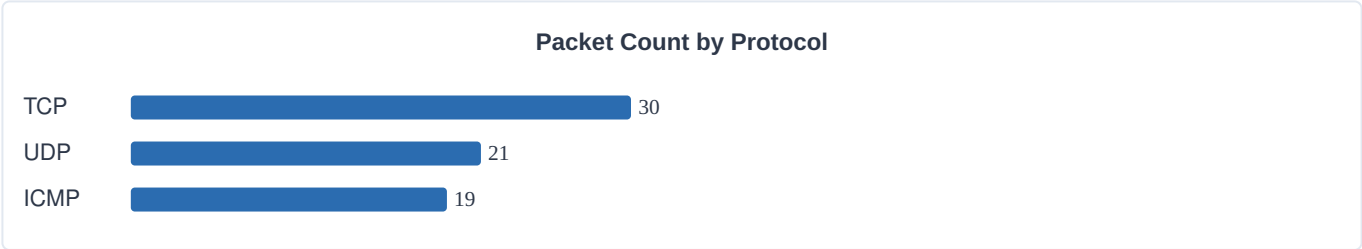


Figure 3: Graphical Metrics Describing Extracted Parameters over Time

Table 1: Logged Simulation Dataset Parameters Sample

Timestamp	Protocol	Source IP	Destination IP	Length
2023-04-21 15:59:16.713506	ICMP	253.14.228.25	30.45.224.111	66
2023-04-21 15:59:16.717113	TCP	123.11.24.239	24.135.93.133	1330

Timestamp	Protocol	Source IP	Destination IP	Length
2023-04-21 15:59:16.727494	UDP	167.20.1.187	8.218.2.217	1454

5.2 Discussion

Let's discuss the importance and prospect of the packet sniffer simulation in additional points! While this simulation is a strong tool for studying network traffic practices and comprehending various protocols, it doesn't generate real network traffic. Therefore, the simulation may not fully describe actual network conditions.

It's also worth mentioning that the recent implementation of the simulation doesn't sustain all possible protocols or packet fields, which suggests that network traffic study is restricted to a subset of gridlock. However, forthcoming work could develop the simulation to possess a wider range of protocols and packet fields, enabling more thorough analysis of network traffic.

Another thrilling possibility for future assignments is the simulation with existing network simulators or real-world network traffic. This would construct more practical scenarios for network analysis and troubleshooting, delivering additional valuable insights. Overall, while the packet sniffer simulation has its restrictions, it's always a useful tool for anyone studying network traffic patterns or operating with network traffic analysis tools. With additional development and integration with other tools and data sources, the simulation's potential could be even greater.

6 Conclusion

The research paper is presenting a packet sniffer in Python language for analysis of network traffic. The code behind give results as random packets, processes them and stores information about packets in a Dataframe so that they can be analysed further. This tool has multiple uses and aspects. This tool can be used for educational purposes, for studying various aspects of network traffic and for testing network traffic analysis tools. There may exist boundaries on number of various implementations, but it provides a simple and accessible beginner point for users likely to be interested in analysis of network traffic.

7 Acknowledgments

As we reflect on our project - "A Python-based Packet Sniffer Simulation for Network Traffic Analysis," we would like to extend our heartfelt gratitude to our mentor, Dr. Ashish Kumar, for his unwavering guidance, support, and invaluable insights. Dr. Kumar's unwavering commitment to excellence and tireless efforts helped us navigate through every obstacle we encountered, ensuring the successful execution of our project. We would also like to express our sincere appreciation to the other professors and friends who aided us in accomplishing our capstone project with flying colors.

Our research paper documents our journey throughout the CSET207 Computer Networks course in the 4th semester of our Bachelor of Engineering in Computer Science program at Bennett University. As we present our work, we hope to inspire fellow students to explore and innovate in the field of computer networking.

Aryan Rekhi
Candidate Signature

Arihant Saxena
Candidate Signature

Ayushman Pranav
Candidate Signature

Bonthu Sai Easwar
Candidate Signature